

PCA para preprocesar datos

Guillermo Ruiz

El algoritmo PCA tiene la particularidad de que sus proyecciones tienden a separar los datos cuando se aplican a bajas dimensiones. En este notebook se verá ese efecto.

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn import decomposition
from sklearn import datasets
```

Leer datos

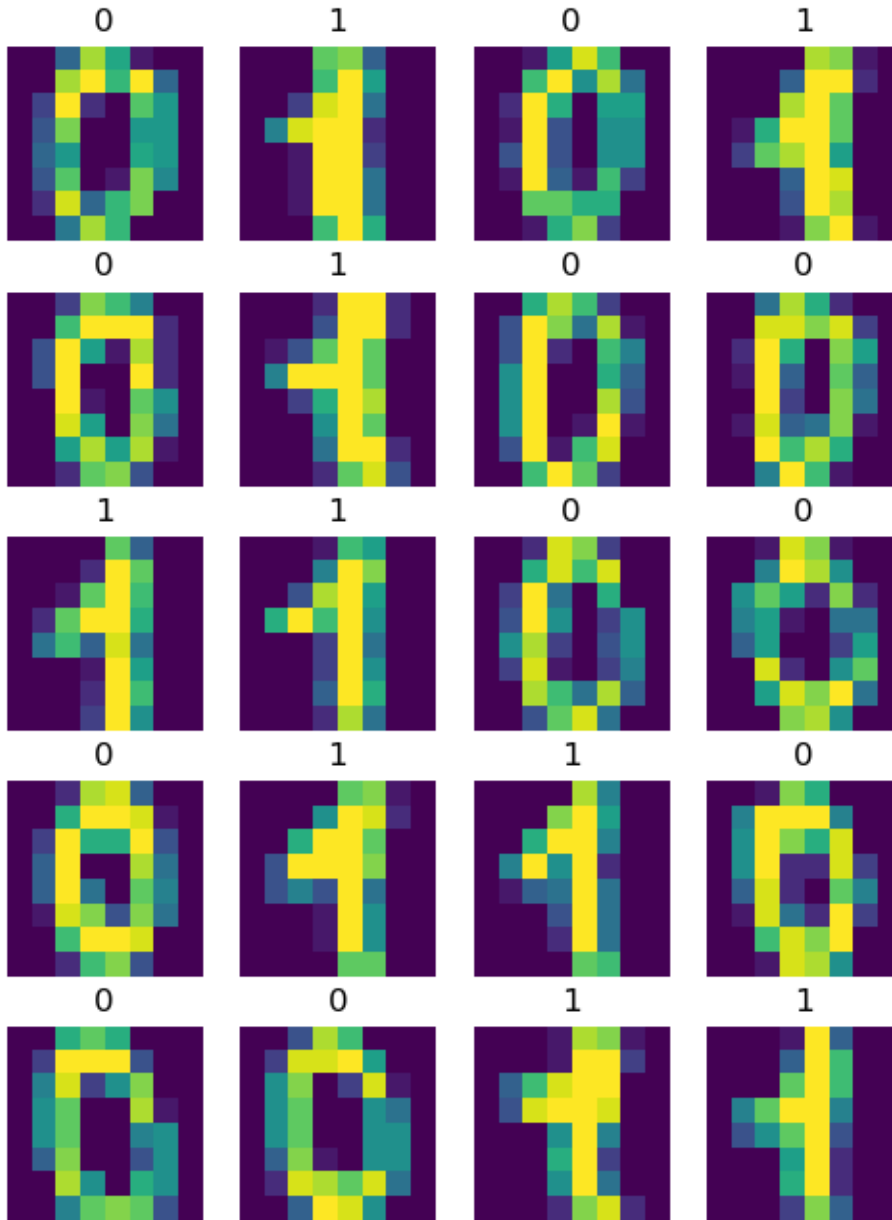
Se lee el conjunto de datos `load_digits` que son imágenes de números escritos a mano. El parámetro `n_class` indica el número de clases que queremos, en este caso sólo la clase de 0 y 1.

```
digits = datasets.load_digits(n_class=2)
X = digits.images.reshape((len(digits.images), -1))
y = digits.target
X.shape, y.shape
```

```
((360, 64), (360,))
```

Mostramos las primeras 20 imágenes, todas son en escala de grises y tienen una resolución de 8x8. La variable `y` contiene el número que hay en la imagen correspondiente.

```
fig=plt.figure(figsize=(6, 8))
columns = 4
rows = 5
for i in range(20):
    img = digits.images[i]
    fig.add_subplot(rows, columns, i + 1)
    plt.imshow(img)
    plt.axis('off')
    plt.title(y[i])
plt.show()
```



Aplicar PCA y graficar en dos dimensiones.

1. Aplicar PCA para reducir la dimensión a dos y graficar los puntos, el color de los puntos debe indicar la clase a la que pertenece.
2. Ahora que los puntos se encuentran separados, aplicar k-means para separar los dos clusters.

3. Graficar otra vez los puntos en dos dimensiones pero donde el color de los puntos indique la pertenencia a uno de los clusters. Comparar con la imagen anterior.

Hacer lo mismo pero dentro de un ciclo `for` donde el número de clases vaya de 2 a 10. Comparar las imágenes conforme se van agregando más clases.